

THE EMOTIONAL RELATIONAL MODEL (ERM)

The Ledger That Turns Human Feeling Into Institutional Intelligence

The Relational Operating System for Emotion

Raheem Asghar - December 2025

1. The Missing Data Model

We have relational schemas for money, inventory, identities, commits, packets, logs, metrics, and events.

We have never had one for **emotion**. Until now.

The Emotional Relational Model (ERM) is the categorical leap: the first time emotion is given **identity**, **lineage**, **referential integrity**, and **state transitions** that can be queried, audited, and replayed.

Everything else in Decipher: SEUs, OUs, PDAM loops, elasticity, memory, agent behavior lives inside this model.

ERM is the ground truth. Everything else is application logic.

2. The Three Structural Laws of the ERM

(Invariants enforced exactly like double-entry bookkeeping or UTXO rules)

1. Identity Law

Each Experience Driver has exactly **one canonical SEU per analysis window**.

This SEU is its emotional account: the single source of truth for its emotional condition.

No duplicates. No parallel interpretations.

2. Lineage Law

Every OU activation is a **timestamped state transition** linked to a SEU, carrying the recorded outcome and consequence.

Transitions are immutable and fully traceable.

3. Memory Law

Every emotional state transition retains referential integrity across time. No state is ever orphaned.

Historical outcomes form the constraints for future states.

These laws are enforced by relational constraints: foreign keys, uniqueness, triggers, and lineage tables.

Once enforced, emotion becomes:

- governable
- auditable
- predictable
- agent-native
- relationally stable

This is the emotional analogue of:

- the relational model (Codd)
- the UTXO ledger (Satoshi)
- the intent model (Stripe)
- the metric + tag model (Datadog)

3. Core Entities and Cardinalities

(The complete ER diagram in nine rows)

All cardinalities are understood "per analysis window" unless otherwise noted. Historical records are kept in full for lineage.

Entity	Description	Key Constraints	Cardinality (core relationships)
experience_drivers	Canonical list of Experience Drivers	UNIQUE(name) or UNIQUE(external_id)	1 driver → 1 active SEU per window; many SEUs across history
seus	Structured Experience Unit (emotional account)	FK(experience_driver_id) → experience_drivers(id) UNIQUE(experience_driver_id, window_id)	1:1 with a driver in a given window
orchestration_units	Mission templates for that SEU (Fix, Optimize, etc.)	FK(seu_id) → seus(id)	1 SEU → many latent OUs
ou_activations	Executed instances of an OU	FK(orchestration_unit_id) → orchestration_units(id) UNIQUE(orchestration_unit_id, activated_at)	1 OU template → many activation instances over time
problem_statements	Extracted, PDCA-ready problem statement	FK(ou_activation_id) → ou_activations(id) UNIQUE(ou_activation_id)	1 OU activation → 1 problem statement
pdca_cycles	Plan–Do–Check–Act iterations for that activation	FK(problem_statement_id) → problem_statements(id)	1 problem statement → many PDCA cycles
outcome_evaluations	Elasticity and outcome assessment per cycle	FK(pdca_cycle_id) → pdca_cycles(id) UNIQUE(pdca_cycle_id)	1 PDCA cycle → 1 outcome evaluation
institutional_memory_events	All lineage events (activations, cycles, drifts, silences)	FK(source_entity_id, source_type) → {above tables}	Many-to-many across time; each event points back to one source row
mentions	Raw unpacked customer mentions grouped into SEUs	FK(seu_id) → seus(id)	Many mentions → 1 SEU

Minimal. Stable. Complete. This schema has not needed revision because the laws are invariant.

4. Why the ERM Is the True Primitive

The SEU is not primitive. The SEU is only the **row** that the ERM laws operate on.

The **model**, the laws, constraints, and lineage is the primitive.

This is the exact inflection point where a system stops being a “CX platform” and becomes **infrastructure**.

It mirrors the same turning point in other categories:

- Bitcoin → once the ledger rules existed, money became programmable
- Stripe → once payment intents existed, commerce became orchestratable
- Git → once the DAG was defined, collaboration became parallelizable
- Snowflake → once compute separated from storage, scale became elastic

ERM is that inflection point for emotion.

5. Properties That Emerge Immediately From the Laws

Property	Enabled By	Enterprise Outcome
Full auditability	Lineage Law	Instant root cause analysis and proof trails
Agent-native state	Identity + Memory	Agents query SQL; no hallucination risk
Elasticity verification	Referential integrity	Know if actions truly improved experience
Silent Guidance	Memory Law	System remembers patterns humans forget
BI compatibility	Pure relational schema	Plug into Tableau/PowerBI instantly
Model independence	LLM not in schema	Swap GPT ↔ Claude freely
Horizontal scaling	Standard DB	Billions of rows without redesign
Zero lock-in	Portable SQL	Organizations own their emotional ledger

These properties require **no extra architecture**. They fall directly out of the three laws.

6. PDAM: The Closed Loop Executed Inside the ERM

-- Perceive

INSERT INTO seus (...) -- Identity Law

-- Decide

INSERT INTO orchestration_units (...)

-- Act

INSERT INTO ou_activations (...) -- Lineage Law

-- Memory

INSERT INTO outcome_evaluations (...)

INSERT INTO institutional_memory_events (...)

PDAM is simply a stored procedure over this schema.

7. Final Declaration

This is not another CX platform. This is the **relational schema emotion has been missing since computing began.**

The Emotional Relational Model is now defined.

The three laws are stable.

The schema is mature.

Every customer-facing institution will eventually operate on this ledger.

Because once emotion gains identity, lineage, and memory enforced at the schema level, themes, dashboards, and sentiment scores become obsolete.

This is not software. This is infrastructure. **And the ledger is now live.**
